MICRO PROJECT REPORT

ADVANCED DATABASE SYSTEMS

SEATSURE: CONCURRENCY-SAFE CAMPUS EVENT REGISTRATION WITH AUTO-PROMOTING WAITLISTS

Short report: the task app, CRUD over REST with React, Express and PostgreSQL

Submitted by:
Bhavya Sharma
University Registration No. 2511021060291

BACHELOR OF TECHNOLOGY
COMPUTER SCIENCE AND ENGINEERING (AI AND FUTURE TECHNOLOGIES)

Under the guidance of
Mr. Vaibhav Giri



Alliance School of Advanced Computing
Alliance University, Bengaluru
October-2026

Live application: https://seatsure-production.up.railway.app
Source code: https://github.com/bhvyashrma17/seatsure

TABLE OF CONTENTS

1. Problem Statement and Approach.....2
1.1 CRUD over REST.....3
1.2 Technologies Used.....3
2. Setup and API Reference.....3
2.1 Running the Project.....3
2.2 API Reference.....4
2.3 Example Requests and Responses.....5
3. The Full Flow: Adding a Task.....5
4. Route Mapping and Validation.....7
4.1 Routes and the Code Behind Them.....7
4.2 Why Validate on Both the Client and the Server.....8
5. Test Cases.....8
5.1 API Test Cases.....8
5.2 Automated Tests.....9
6. Screenshots.....9
7. Conclusion.....10

1. Problem Statement and Approach

Students juggle coursework, club activities and errands. Notes on paper or in a chat app get lost, cannot be opened on both a phone and a laptop, and accept anything, including empty or half-written entries. The task for this project was to build a small full-stack application in which a user can **create, view, update and delete tasks**: a React interface, a REST API in Node.js and Express that validates its input and answers with correct status codes, and a database that keeps the data across restarts.

I built this as part of **SeatSure**, the campus event registration system of my integrated micro project. Every user has a personal to-do list, **My tasks**: they can add a task with optional notes and a due date, see open and finished tasks with counts, edit a task, tick it done (or undo that) and delete it. The tasks are stored in PostgreSQL. The rest of SeatSure (events, registrations and waitlists) uses the same layers and is described in the detailed report.

For Advanced Database Systems, the parts to read most closely are the SQL behind each route (Section 4.1) and the constraints that protect the tasks table (Section 4.2). The detailed report for this course, `SeatSure_DetailedReport_AdvancedDatabase_BhavyaSharma.pdf`, covers the course concepts in depth.

1.1 CRUD over REST

REST treats data as *resources* with their own URLs: `/api/v1/tasks` is the collection of the current user's tasks and `/api/v1/tasks/:id` is one task. The HTTP method says which operation to perform, the body carries JSON, and the status code tells the client what happened, so the four CRUD operations need no verbs in the URL (Table 1.1). Every request stands on its own: it names the user in the `X-User-Id` header, and the server keeps no session between requests.

Table 1.1: The four CRUD operations as HTTP requests on the tasks resource

Operation	HTTP request	Success response	Typical errors
Create	`POST /api/v1/tasks` with `{title, notes?, dueDate?}`	201 Created, a `Location` header and the new task	400 invalid input
Read	`GET /api/v1/tasks` or `GET /api/v1/tasks/:id`	200 OK and the task list or one task	404 missing id
Update	`PATCH /api/v1/tasks/:id` with the changed fields	200 OK and the updated task	400, 404
Delete	`DELETE /api/v1/tasks/:id`	204 No Content	404 missing id

1.2 Technologies Used

Table 1.2: Technologies and what each is used for

Part	Technology	Used for
Frontend (`frontend/`)	React 19 with Vite 8, React Router 6, TanStack Query 5, Axios 1, React Hook Form 7	Pages and routing, caching server data, HTTP requests, forms and validation in the browser
Backend (`backend/`)	Node.js 22, Express 5, express-validator 7	The REST API, its middleware and validation on the server
Database (`db/`)	PostgreSQL 18 with Prisma 7 (MongoDB stores the event notifications; tasks do not use it)	Tables and constraints, migrations, seed data and queries
Testing	Vitest 5, Supertest 7, React Testing Library	Automated database, API and component tests
Deployment	Railway and GitHub	The live application and the public source code

2. Setup and API Reference

2.1 Running the Project

The project needs Node.js 22 and PostgreSQL 18. MongoDB is optional: in development an in-memory server starts by itself. The submission folder contains `frontend/`, `backend/`, `db/` and `docs/`. From that folder:

1. `npm install` installs the three workspaces (`frontend`, `backend`, `db`).
2. Copy `.env.example` to `.env` and `frontend/.env.example` to `frontend/.env`, then set `DATABASE_URL` (and `DATABASE_URL_TEST` for the tests) to your PostgreSQL user.
3. `npm run setup` creates the database, applies the migrations and loads the seed data: 230 users, 30 events and 46 tasks. Alternatively, restore the dump with data: `createdb seatsure_dev && psql -d seatsure_dev -f db/seatsure_dump.sql` (the schema alone is in `db/schema.sql`).
4. `npm run dev` starts the API on http://localhost:4000 and the React app on http://localhost:5173.
5. Open http://localhost:5173, choose a demo user in the header (for example Priya Singh) and open **My tasks**.
6. `npm test` runs the 153 automated tests; `npm run test:api` sends the live API checks while the API is running.

The deployed copy at https://seatsure-production.up.railway.app works the same way without installing anything.

2.2 API Reference

Every route is under `/api/v1` and needs the `X-User-Id` header; without it the API answers 401. Responses share one JSON envelope: `{status, data, meta?}` on success and `{status, code, message, errors?, requestId}` on failure (Table 2.1). A task is returned as `{id, title, notes, dueDate, completed, completedAt, createdAt, updatedAt}`. The routes for events and registrations are listed in the README.

Table 2.1: The tasks API (paths relative to `/api/v1`)

Method and route	Request body	Success	Errors
`GET /tasks ?status=all|open|done`	none	200: tasks, open ones first, and `meta: {total, open, done}`	400 bad filter, 401
`GET /tasks/:id`	none	200: one task	404 `TASK_NOT_FOUND`
`POST /tasks`	`{title, notes?, dueDate?}`	201: the new task, `Location: /api/v1/tasks/:id`	400 `VALIDATION_ERROR`, 401
`PATCH /tasks/:id`	any of `{title, notes, dueDate, completed}`	200: the updated task	400 `VALIDATION_ERROR` or `EMPTY_UPDATE`, 404
`DELETE /tasks/:id`	none	204: no body	404 `TASK_NOT_FOUND`

Another user's task is answered with 404, exactly like a missing one, so the API does not reveal which ids exist.

2.3 Example Requests and Responses

Listing 2.1 shows the three example cases from the course guidelines, sent to the running API with `curl` on 27 October 2026. Headers that do not matter here are left out.

Listing 2.1: Create, invalid input and missing id: requests and the real responses

```
$ curl -i -X POST localhost:4000/api/v1/tasks -H "X-User-Id: 11" \
       -H "Content-Type: application/json" -d '{"title":"Buy notebook"}'
HTTP/1.1 201 Created
Location: /api/v1/tasks/49
{"status":"success","data":{"id":49,"title":"Buy notebook","notes":null,
 "dueDate":null,"completed":false,"completedAt":null,
 "createdAt":"2026-10-27T04:24:20.965Z","updatedAt":"2026-10-27T04:24:20.965Z"}}

$ curl -i -X POST localhost:4000/api/v1/tasks -H "X-User-Id: 11" \
       -H "Content-Type: application/json" -d '{"title":""}'
HTTP/1.1 400 Bad Request
{"status":"fail","code":"VALIDATION_ERROR","message":"Some fields are invalid",
 "errors":[{"field":"title","message":"Title is required","location":"body"}],
 "requestId":"b2c96f9b-9177-4901-b5ae-17d6524f7a29"}

$ curl -i -X DELETE localhost:4000/api/v1/tasks/99 -H "X-User-Id: 11"
HTTP/1.1 404 Not Found
{"status":"fail","code":"TASK_NOT_FOUND","message":"Task 99 not found",
 "requestId":"bea394fc-e670-465d-8c42-c1ab8ba53498"}
```

3. The Full Flow: Adding a Task

Figure 3.1 follows one request, adding the task “Buy notebook”, from the browser to the database and back to the screen. The numbers in brackets below refer to the arrows.

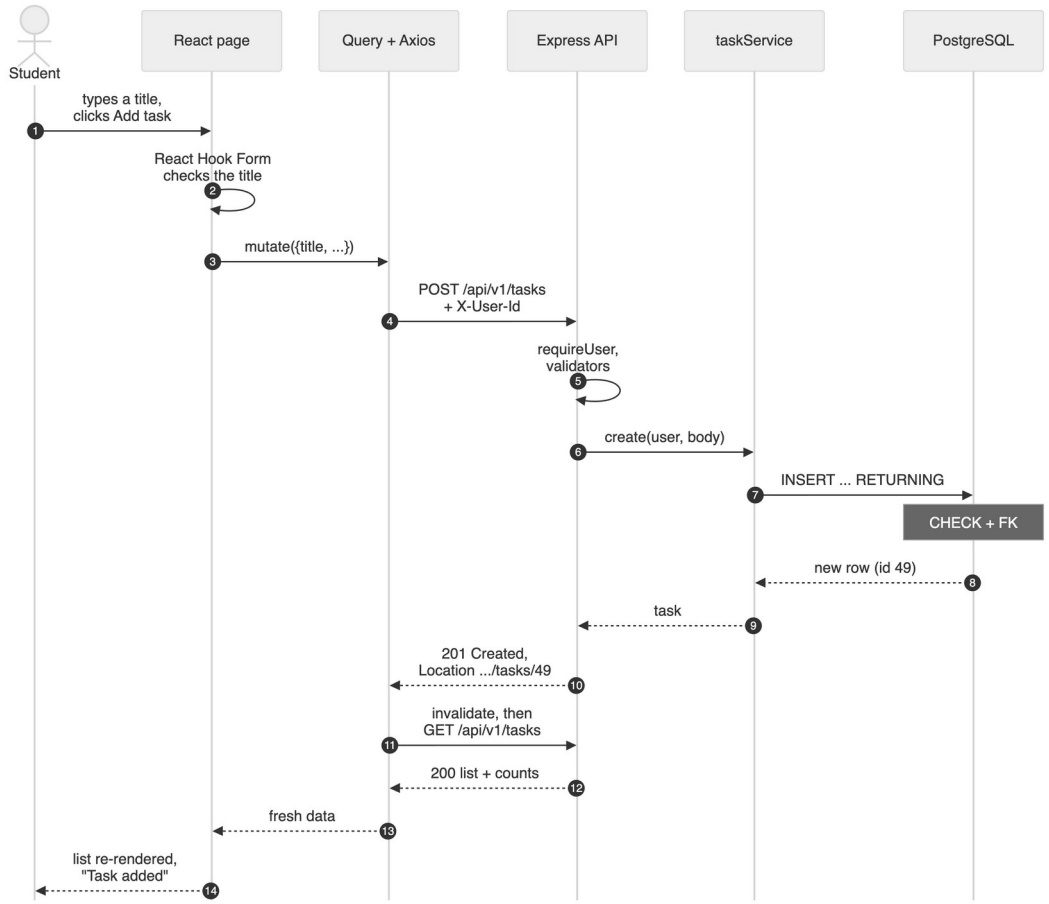


Figure 3.1: Adding a task: from the form to PostgreSQL and back to the screen

1. **In the browser (1–2).** The user types a title in the "Add a task" form. React Hook Form checks it before anything is sent: an empty title shows "Title is required" under the field.
2. **Request (3–4).** The form calls the `useCreateTask` mutation of TanStack Query, which sends `POST /api/v1/tasks` with the JSON body `{title, notes, dueDate}` through a shared Axios instance; an interceptor adds the `X-User-Id` header.
3. **API (5–6).** Express runs its middleware (request id, timer, CORS, a JSON parser limited to 10 kB) and then the tasks router: `requireUser` loads the user (401 if there is none), the express-validator chain trims and checks the fields and keeps only the declared ones (400 with a list of field errors if anything is wrong), and a thin controller calls `taskService.create`.
4. **Database (7–8).** The repository uses Prisma to run `INSERT INTO tasks … RETURNING`. PostgreSQL checks its own rules (the title is not blank and the user exists) and returns the new row.

5. **Response (9–10).** The controller answers **201 Created** with `Location: /api/v1/tasks/49` and the task as JSON. An error thrown at any step is turned into the same error envelope by one error-handling middleware at the end of the pipeline.
6. **Display (11–14).** The mutation marks the cached task list as stale, so TanStack Query fetches `GET /api/v1/tasks` again. React re-renders the list with the new task and new counts, and a "Task added" message appears.

Updating and deleting take the same path with `PATCH` and `DELETE`. Ticking a task done is *optimistic*: the tick appears at once, before the server answers; if the server refuses, the previous list is put back and an error message is shown.

4. Route Mapping and Validation

4.1 Routes and the Code Behind Them

Each route maps to one CRUD operation and to one function in each layer (Table 4.1). The router itself is a short list of method, path, validation chain and handler (Listing 4.1).

Table 4.1: Route → controller and service → SQL (all routes run `requireUser` first)

CRUD	Route	Controller → service	SQL run through Prisma	Status
Create	`POST /tasks`	`create` → `taskService.create`	`INSERT INTO tasks … RETURNING`	201
Read	`GET /tasks`	`list` → `taskService.list`	`SELECT … WHERE user_id = $1 ORDER BY completed, due_date NULLS LAST, created_at`, plus two counts	200
Read	`GET /tasks/:id`	`get` → `taskService.get`	`SELECT … WHERE id = $1 AND user_id = $2`	200, 404
Update	`PATCH /tasks/:id`	`update` → `taskService.update`	the same ownership check, then `UPDATE tasks SET … WHERE id = $1`	200, 404
Delete	`DELETE /tasks/:id`	`remove` → `taskService.remove`	the same ownership check, then `DELETE FROM tasks WHERE id = $1`	204, 404

Listing 4.1: The CRUD routes for tasks (`backend/src/routes/tasks.routes.js`, lines 7–16)

```
 7  export function createTasksRouter({ tasks, auth }) {
 8    const router = express.Router();
 9    router.use(auth.requireUser); // tasks are personal, so every route needs a
       known user
10    router.get('/', listTasksRules, validate, tasks.list);
11    router.post('/', createTaskRules, validate, tasks.create);
12    router.get('/:id', taskIdRules, validate, tasks.get);
13    router.patch('/:id', updateTaskRules, validate, tasks.update);
14    router.delete('/:id', taskIdRules, validate, tasks.remove);
15    return router;
16  }
```

4.2 Why Validate on Both the Client and the Server

The same rules are checked in the browser, in the API and in the database (Table 4.2). Each layer has its own reason for being there.

Table 4.2: Where each rule for a task is enforced

Rule	Browser (React Hook Form)	API (express-validator)	PostgreSQL
Title is required	`v.trim().length > 0`	`.trim().notEmpty()`: 400 "Title is required"	`CHECK (length(btrim(title)) > 0)`
Title: at most 120 characters	`maxLength: 120`	`isLength({ max: 120 })`	`VARCHAR(120)`
Notes: at most 500 characters	`maxLength: 500`	custom check, 500	`VARCHAR(500)`
Due date is a date	`<input type="date">`	`YYYY-MM-DD` and a real date	`DATE` column
`completed` is true or false	checkbox	`isBoolean({ strict: true })`	`BOOLEAN`; `CHECK (completed = (completed_at IS NOT NULL))`
The task belongs to the user	user chosen in the header	owner taken from `X-User-Id`, never from the body; another user's task gives 404	foreign key to `users`, `ON DELETE CASCADE`

- **Validation in the browser is for the user.** A mistake is shown under the field at once, without a round trip, and no request is wasted on input that cannot succeed.
- **Validation on the server is for safety.** The browser is under the user's control: anyone can skip the form and send a request with `curl`, Postman or the browser's developer tools, as live check API-02 does. Every request has to pass through the API, so it checks every field again, drops fields it does not expect (a client cannot set `userId` or `completedAt`), and answers 400 with a list of the invalid fields, which the form then shows under the matching input.
- **Database constraints are the last line of defence.** If a bug or another program ever goes around the API, PostgreSQL still refuses a blank title or a finished task without a completion time.

5. Test Cases

5.1 API Test Cases

`npm run test:api` sends real HTTP requests to the running API and records the expected and the actual result of each. Table 5.1 lists the 7 task cases from the run on 27 October 2026; the

first three are the examples given in the course guidelines. All 7 passed, and so did the other 14 checks of the same run, which cover events and registrations (21 of 21).

Table 5.1: API test cases for tasks: expected and actual results

Test	Case	Request	Expected	Actual	Result
API-01	Create	`POST /api/v1/tasks {title:"Buy notebook"}`	201 new task	201 id 48, completed false, Location /api/v1/tasks/48	Pass
API-02	Invalid input	`POST /api/v1/tasks {title:""}`	400 error	400 VALIDATION_ ERROR: Title is required	Pass
API-03	Missing ID	`DELETE /api/v1/tasks/99`	404 not found	404 TASK_NOT_FOUND	Pass
API-04	Read	`GET /api/v1/tasks`	200, my tasks incl. "Buy notebook"	200 6 tasks (5 to do, 1 done)	Pass
API-05	Update (toggle)	`PATCH /api/v1/tasks/48 {completed:true}`	200, done with completedAt	200 completed true, completedAt set	Pass
API-06	Update (edit)	`PATCH /api/v1/tasks/48 {title:"Buy two notebooks"}`	200, new title	200 "Buy two notebooks"	Pass
API-07	Delete	`DELETE /api/v1/tasks/48`	204 No Content	204 No Content	Pass

5.2 Automated Tests

Three automated suites run with `npm test`; all 153 tests pass (Table 5.2). Fifteen of them are about tasks.

Table 5.2: Automated test suites (27 October 2026)

Suite and command	Tests	The task tests and what they check	Result
Database `npm run test:db`	50	DB-C14 – C16: a blank title and a finished task without a completion time are rejected; a user's tasks are deleted with the user	50 passed
Backend `npm run test:backend`	71	BE-I27 – I33: 201 with `Location`; 400 for an empty title; 204, then 404 for a second delete; another user's task gives 404; 400 for an empty update or a bad date; 401 without a user	71 passed
Frontend `npm run test:frontend`	32	FS-T28 – T32: list and counts; the title is checked before sending; an optimistic tick rolls back on failure; delete; a server error appears under the field	32 passed
Total	**153**	**15 task tests**	**153 passed**

6. Screenshots

Figure 6.1 shows the four operations in the running application, and Figure 6.2 shows the validation case. More screenshots, including the event pages, are in `docs/screenshots/`.

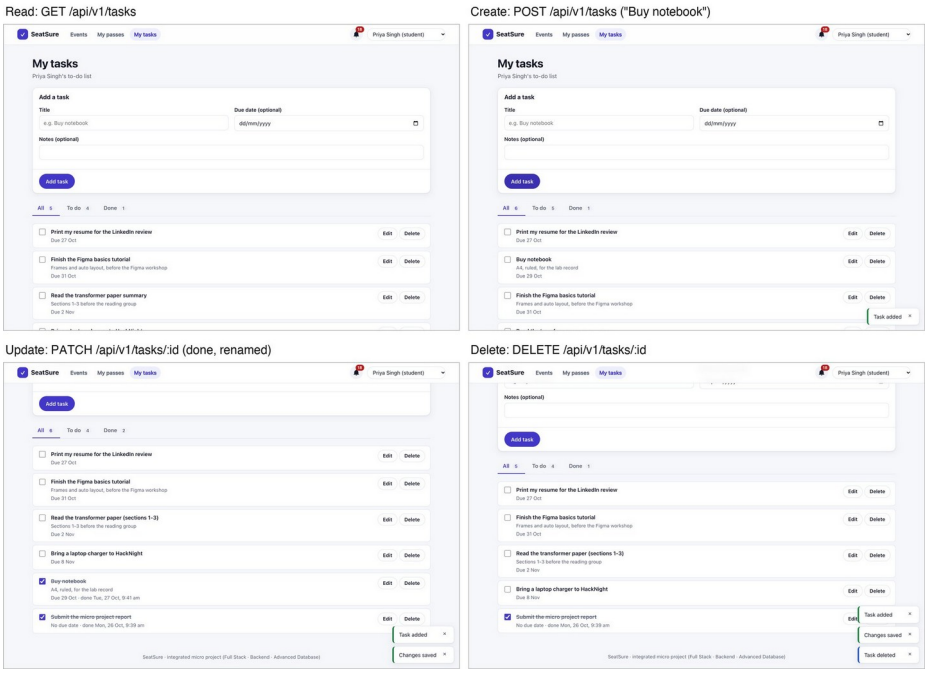


Figure 6.1: My tasks: read (the list), create ("Buy notebook" added), update (ticked done, another task renamed) and delete

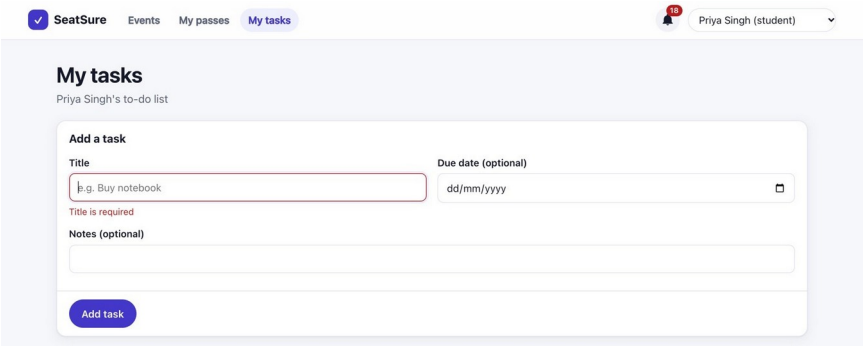


Figure 6.2: Validation in the browser: an empty title is stopped with "Title is required" (the API would answer 400)

7. Conclusion

The task feature meets the brief: a user can create, view, update and delete tasks in a React interface; an Express REST API validates every request and answers with the right status codes (201, 200, 204, 400, 404 and 401); and PostgreSQL keeps the tasks and the seed data across restarts. The same rules are checked in the browser, the API and the database, and 153 automated tests and 21 live API checks pass.

The main limitation is identity: the user is picked from a list of demo users and sent in a header, because login comes later in the course, so adding real authentication is the next step. The application is live at https://seatsure-production.up.railway.app and the code is at https://github.com/bhvyashrma17/seatsure.